



3



8



960 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Mpi & Pvm

د. روان قرعوني



RB Informatcs; 14/05/2025

محتوى مجاني غير مخصص للبيع التجاري

برمجة تفرعية

السلام عليكم ورحمة الله وبركاته

هذه المحاضرة مهمة جداً بالامتحان

- ما هو ال PVM؟
- برنامج عبارة عن Parent Childreng أو master و slave
- عند عمل run للبرنامج فإن أول من يتم تنفيذه هو ال Parent(Master) بعدها يقوم ال master بإنشاء PVM spawn والتي تقوم بإنشاء الأبناء أي إنشاء مهام وكل مهمة تنفذ كود معين
- يوجد لتعليمة ال PVM spawn عدة Parameters ما يهمنا منها بالنظري:
- 1-Parameter : المصفوفة
- 2- وماذا يعيد : int x: مثلاً
- بالنسبة لآخر Parameter تكون المصفوفة هي by reference يمكن التعديل عليها بحيث يكون فيها id الأبناء (task id) ويكون رقم ثابت نحتاجه لاحقاً للتواصل مع المهمة والتخاطب مع المهام
- خرج ال PVM Spawn : يكون عدد الأبناء التي تم إنشاؤها بنجاح
- مثال : يوجد مصفوفة 10*10 نريد توزيعها على 4 أبناء أي لدي 4 كتل كل منها لمهمة فيكون لدينا أب وثلاث أبناء لو كان خرج ال Pvm Spawn ابان لنتج خطأ (لضرب الكود) وذلك لوجود ابن تم إرساله لرقم عشوائي غير موجود
- لذلك يجب ان يكون هناك جزء من الكود يتحقق من عدد الأبناء

```
numtids = pvm_spawn(taskname, argv, 0, "", 10, tids);

if (numtids == 10) {
    printf("Good , you can continue\n");
} else {
    fprintf(stderr, "Error: Not enough children were created.\n");
    pvm_exit();
    return 1;
}
```

توابع الإرسال والاستقبال

- بعد إنشاء الأبناء (المهام):
نتعامل مع التواصل (سواء مع الاب فقط ، أو بين بعضهم) . لكل حالة طريقة تعامل . كل ابن يجب أن يعلم ما المهمة التي قبله وماهي التي تليه.
- يُعد كل ابن عبارة عن ملف exe يتخاطب مع ملفات exe أخرى -تمثل الأبناء- عن طريق pvm
- تتم عملية التخاطب أي التواصل بين الأبناء مع بعضها بواسطة توابع الإرسال والاستقبال التي هي Pvm_send و Pvm_receive لكل منها 3 خطوات:

■ خطوات الإرسال Pvm_send :

- 1-تهيئة الرسالة : Pvm_initsend() مثل: ابن يريد الإرسال لابن آخر
- 2- تحزيم الرسالة : يتم تحزيم المعلومة حسب الطريقة المناسبة بحيث يوجد 3 أنواع لتوابع التحزيم نختار منها حسب نوع البيانات الذي أريد تحزيمه pvm_pkstr() و pvm_pkfloat() و pvm_pkint()
- 3-إرسال الرسالة: Pvm_send() يأخذ 2-Arguments
 - Destination task id
 - Message tag

■ تذكير: يجب مقابلة كل تابع send بتابع recv لضمان عدم توقف البرنامج

■ خطوات الاستقبال Pvm_receive :

■ (Buff Info) خطوة يمكن القيام بها لقراءة المعلومة

- 1-ينتظر حتى تأتي رسالة : Pvm_recv() يأخذ 2-Arguments

■ Source task Id

■ Message tag

- 2- قراءة معلومات ال Buffer : Pvm_bufinfo()

- 3- فك تحزيم الرسالة: عن طريق أحد التوابع حسب نوع البيانات pvm_upkfloat() و pvm_upkstr() و pvm_upkint()



ملاحظة:

يستخدم التابع pvm_bufinfo() لتمييز الرسالة المرادة من بين عدة رسائل موجودة , فيمكن تجاهل استخدامه في حال التأكد من استلام الرسالة الصحيحة . مثلا في حالة عدم وجود رسائل أخرى.
هكذا نكون قد أمنا عملية التخاطب بين ملفات ال exe

pvm_mytid()

قيمة خرجها هي ال task id الخاص بل ابن الذي يستدعي التابع .

pvm_parent_tid()

قيمة خرجها هي ال task id الخاص ب الأب للابن الذي استدعى التابع .

التوابع التجميعية

1. متى يمكن استخدام هذه التوابع؟

عندما نحتاج الى group وكل group يحوي مجموعة tasks معينة

2. الفائدة: تكمن فائدة إنشاء group واستخدام التوابع التجميعية في تهيئة الرسالة المراد إرسالها لمرة واحدة فقط

3. كيف يتم إنشاء ال group :

في ال Pvm لا يوجد group افتراضي هذا يعني أن مهمة إنشاء ال group تكون مسؤولية المبرمج وذلك عن طريق pvm join group وإن أول مهمة تصل لل group join تقوم بإنشاء group وال tasks الاخرى التي تصل لنفس التعليمة تدخل لنفس ال group ولا تقوم بإنشاء group جديد (شرط ان يكون لل group نفس الاسم) Pvm_join_group("ABC")

توضيح: في حال كان لدي 10 مهام قمنا باستدعاء التابع Pvm_join_group("ABC") في 6 مهام منها أول مهمة تقوم بتنفيذ التابع تقوم بخلق ال group و تقوم المهام الخمسة المتبقية بالانضمام لنفس المجموعة و تأخذ كل مهمة رقم خاص بها ضمن المجموعة

من توابع الارسال أيضا يوجد نوعان

- Pvm_bcast(): ترسل رسالة لجميع المهام بال group ومن ضمنهم نفسها
- Pvm_mcast(): لا تستخدم بال group بل تستخدم لإرسال الرسائل ل task ids معينة فهي تشبه ال send وتحتوي في احدى ال parameters مصفوفة task id بالمهام التي تريد ان ترسل لهم إذا فهو يرسل رسالة لمجموعة من المهام حسب مصفوفة ال ids

عليك بالتأني وأعط دون مني وسر نحو اثريا بسير مطمئن لا لن تنال فوزاً وعزاً

بالتمني فلا تكن كسولاً ولا تقل لو أنني



الفرق بين الـ mpi و pvm

mpi	pvm
تعد interface لها عدة إصدارات تتشابه بالجوهر مع بعض الاختلافات لذلك نستطيع استخدام mpi بدون الحاجة لوجود virtual machine	تعد الـ pvm مكتبة جاهزة لها إصدار ثابت للجميع
في الـ mpi عند بداية تنفيذ التطبيق تظهر واجهة لتحديد عدد الأبناء اللازمة لكل مهمة . فلا يوجد مفهوم أب - ابن	في pvm تكون عملية إنشاء الأبناء عند الحاجة أثناء التنفيذ. بعد تنفيذ المهمة الأب
في الـ mpi عند بداية التنفيذ جميع المهام المنشئة تصبح ضمن مجموعة أساسية تدعى "MPI_COMM_WORLD" (محيط الاتصال العالمي)	- في pvm عند بداية التنفيذ تكون المهام المنشئة مستقلة (لا يوجد مجموعة افتراضية تجمع بينهم) ويتم استخدام pvm_join() لوضع المهام ضمن الـ group
هنا اسم المجموعة يدعى "communicator"	تستخدم مفهوم الـ group

ملاحظة:

- يمكن إنشاء مجموعات في الـ mpi عن طريق إنشاء communicator يضم عدد معين من المهام وسيكون مجموعة فرعية ضمن المجموعة الأساسية "MPI_COMM_WORLD"

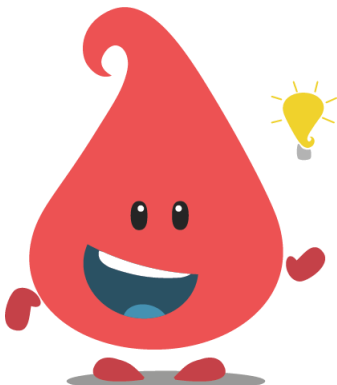
- في الـ mpi لا نقوم بوصف المهام على أنها أبناء لأن جميع المهام متساوية

من أجل أن يعمل برنامج الـ mpi نستخدم :

1. mpi_init() يبدأ البرنامج به دائما

2. mpi_finalize() ينتهي به دائما

لا يوجد تحريم بل فقط ارسال واستقبال



اهم تابعين لدينا في ال mpi أي يجب استخدامهم دائماً بأي كود

1-MPI_Comm_rank ()

يقوم بإعادة الrank (ترتيب المهمة ضمن الCommunicator) يبدأ ترتيب أول مهمة من الصفر، كل مهمة تدخل تأخذ Rank جديد ، ال Rank فيه يشبه عمل الId في الpvm

ملاحظة: لا يوجد أب يعني ان كل الموجود عبارة عن مهام لكن يوجد مهمة root تجمع كل المهام وأنا من أقوم بتحديددها (أي احدد المهمة التي سوف ترسل وتُجمّع)

2-MPI_Comm_size ()

خرجه هو عدد المهام في ال Communicator

توابع الارسال والاستقبال في حالة التنفيذ المتزامن

1-MPI_Send()

2-MPI_Recv

توابع الارسال والاستقبال في حالة التنفيذ غير المتزامن

1-MPI_Isend()

2-MPI_Irecv()

MPI_Waitall()

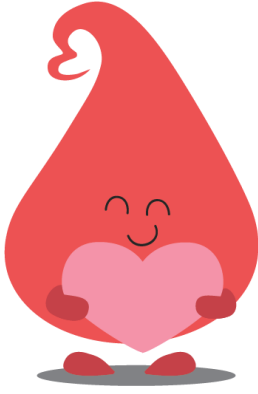
يستخدم في حالة التنفيذ غير المتزامن لجعل كل المهام غير المتزامنة تجتمع حتى تنتهي له عدة parameters أهمها مصفوفة ال request التي تحدد طلبات الارسال التي يجب انتظارها

MPI_Barrier()

يشبه الMPI_Waitall() لكنه يستخدم لمزامنة كل المهام داخل ال communicator (في ال MPI_COMM_WORLD او في group)

كل مهمة تتوقف (يتم عمل pause لها ولا تتابع أي مهمة عملها حتى تصل جميع المهام لنقطة التزامن المحددة)
مكانه:

يجب أن يتم كتابته خارج أماكن الشروط (if,else) لأنه يجب على كل المهام أن تمر عليه مثال لاستخدامه: في حال أردت ضمان استلام البيانات قبل إرسال بيانات أخرى تعتمد على وجود البيانات الأولى



يوجد توابع تجميعية يمكن تنفيذها في ال Communicator

- MPI_Gather
- MPI_Scatter
- MPI_Reduce
- MPI_Bcast

MPI_Reduce_scatter

ليكن لدي 4 مهام

كل مهمة تريد إرسال مصفوفة بطول 4 عناصر إلى المهام (المهام المتبقية ونفسها)

نقوم بتطبيق scatter كما تعلمنا في المحاضرة السابقة

عندها سيتم إرسال القيم عند ال $index = 0$ في كل من المصفوفات الأربعة للمهمة الأولى

والقيم عند ال $index = 1$ في المصفوفات الأربعة للمهمة الثانية

والقيم عند ال $index = 2$ في المصفوفات الأربعة للمهمة الثالثة

والقيم عند ال $index = 3$ في المصفوفات الأربعة للمهمة الرابعة

هكذا نقوم قد طبقنا الإرسال باستخدام scatter

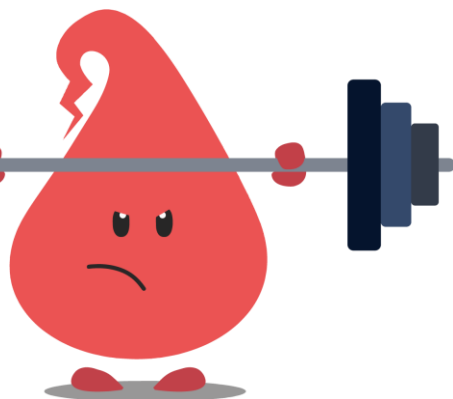
بعد ان تستلم كل مهمة القيم المُرسلة لها تقوم بتطبيق reduce عليها , أي تقوم بتنفيذ عملية معينة على القيم المُستلمة .

ليكن لدينا المثال التالي:

T1	T2	T3	T4
0	0	0	0
1	1	1	1
2	2	2	2
3	3	3	3

بعد تطبيق reduce وبفرض كانت العملية هي الجمع
ينتج

T1	T2	T3	T4
0	4	8	12



سؤال عند تنفيذ ال reduce هل أستطيع تنفيذ عملية مختلفة في كل مهمة ؟

لا يجب أن تقوم جميع المهام بتنفيذ نفس العملية (جمع , or , and...)

- MPI_AlltoAll()
- ليكن لدينا 4 مهام وكل مهمة تريد إرسال مصفوفة بطول 4 عناصر للمهام (المهام الأخرى و المهمة نفسها)
- كما في التابع السابق تقوم المهام أولاً بعملية إرسال باستخدام scatter , بعد تنفيذ الإرسال يصبح لدي في كل مهمة مصفوفة جديدة .

مثال:

T1	T2	T3	T4
0	5	9	13
1	6	10	14
2	7	11	15
3	8	12	16



1	2	3	؟
5	6	7	8
9	10	10	12
12	15	10	16

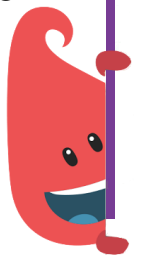
T1	T2	T3	T4
1	2	3	4

يمكن تلخيص عمله بأنه يقوم بقلب السطر إلى عامود و العامود إلى سطر

نلاحظ أن التابع MPI_AlltoAll يقوم بنفس عمل التابع MPI_Reduce_scatter لكن بدون تنفيذ أي عملية على الرسائل المُستلمة

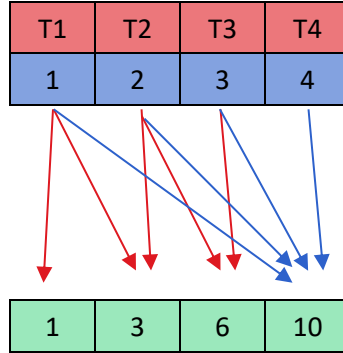
- MPI_Scan ()
- ليكن لدينا 4 مهام وكل مهمة تملك قيمة معينة تقوم كل مهمة بالقيام بعملية -تحدد ضمن بارامترات التابع- على القيمة الموجودة لديها و القيم الموجودة لدى المهام التي قبلها !!

عندما تطلب من الله الزهور لا تتفاجأ إذا أرسل المطر أولاً



للتوضيح

تقوم المهمة الأولى بتنفيذ العملية على القيمة التي لديها فقط لأنه لا يوجد مهام قبلها بينما تقوم المهمة الثانية بتنفيذ العملية على قيمتها وعلى القيمة الموجودة في المهمة الأولى (قبل التعديل) أما المهمة الثالثة تقوم بتنفيذ العملية على كل من قيمتها وقيم المهمة الأولى والثانية



تعد التوابع الثلاث السابقة توابع تجميعية متقدمة.

سؤال ما الفائدة من هذه التوابع التجميعية المتقدمة؟

سنجد حالات معينة تقوم هذه التوابع بتوفير الوقت اللازم لكتابة أكثر من أمر إرسال كما يوجد العديد من التوابع الأخرى المشابهة.

إلى اللقاء في المحاضرة القادمة